

Лекции по типам данных в C#

С примерами применения и шпаргалкой терминов

ЛЕКЦИЯ 1. Что такое тип данных и зачем он нужен

Определение. *Тип данных* — это описание того, **какие значения** может хранить переменная или поле, **сколько памяти** они занимают и **какие операции** с ними можно выполнять.

Зачем это нужно. Компилятор C# — строго типизированный. Это значит: если переменная объявлена как `int`, в неё нельзя записать строку. Это защищает от огромного класса ошибок — тех, что в языках без типизации (Python, JavaScript) проявляются только в момент выполнения.

Пример — что даёт типизация. Допустим, мы пишем:

```
product.Price = "дорого";
```

Компилятор выдаст ошибку: *cannot convert string to double*. В Python такая строка сломала бы программу, когда кто-то попытался бы посчитать `Price * Quantity`. C# ловит ошибку **до запуска**.

Две большие группы типов в C#:

1. **Значимые (value types)** — хранят само значение. Пример: `int`, `double`, `bool`, `char`.
2. **Ссылочные (reference types)** — хранят ссылку на объект в куче. Пример: `string`, массивы, классы (`Product`, `Customer`).

Про разницу — Лекция 8.

ЛЕКЦИЯ 2. Целочисленные типы

2.1. `int` — основной целый тип

Определение. `int` (*integer*) — целое число от $-2\,147\,483\,648$ до $2\,147\,483\,647$. Занимает 4 байта.

Зачем нужен. Для всего, что по смыслу целое: количество, возраст, номер заказа.

Пример.

```
public int Quantity;    // поле в классе Product
public int Age;         // поле в классе Customer
```

Что можно делать.

```
product.Quantity = 5;
customer.Age = 25;
int total = product.Quantity + 3;    // 8
int doubled = product.Quantity * 2; // 10
```

Что нельзя.

```
product.Quantity = 2.5;    // ошибка: нельзя double в int без явного преобразования
product.Quantity = "пять"; // ошибка: строка в int
```

Когда выбирать `int`. По умолчанию для целых чисел — почти всегда `int`. Это самый быстрый и самый удобный целый тип в .NET.

2.2. `long` — длинное целое

Определение. `long` — целое от $-9.2 \cdot 10^{18}$ до $9.2 \cdot 10^{18}$. Занимает 8 байт.

Зачем нужен. Когда `int` не хватает: количество пользователей в большой системе, идентификаторы в БД, временные метки.

Пример.

```
public long OrderNumber;  
orderNumber = 9_000_000_000; // в 4 раза больше, чем вмещает int
```

2.3. short , byte , sbyte

Определение.

- short — 2 байта, от -32 768 до 32 767.
- byte — 1 байт, от 0 до 255.
- sbyte — 1 байт, от -128 до 127.

Зачем нужны. Экономия памяти в огромных массивах. В учебных задачах не нужны — используй `int`.

2.4. uint , ulong — беззнаковые

Определение. Те же `int` и `long`, но **без отрицательных значений**, зато с удвоенным максимумом.

Зачем. Практически никогда в обычном коде. В C# для денег и количества лучше `int` — если случайно уйдёт в минус, компилятор не будет ругаться, но ты увидишь ошибку в логике.

ЛЕКЦИЯ 3. Дробные типы

3.1. double — основной вещественный тип

Определение. `double` (*double precision floating point*) — число с плавающей точкой двойной точности. Занимает 8 байт. Примерно 15 значащих цифр.

Зачем нужен. Для физических величин, координат, процентов, где не критична точность до копеек.

Пример.

```
public double Price;      // цена товара  
public double Balance;    // баланс покупателя
```

```
public double Percent;    // размер скидки в процентах
public double Rating;     // рейтинг магазина
```

Что можно делать.

```
product.Price = 99.99;
store.Rating = 4.7;
double discounted = product.Price - product.Price * 0.1;
```

Важно — точность. `double` **нельзя** использовать для денег, где важна точность до копейки.

Пример:

```
double a = 0.1 + 0.2;
Console.WriteLine(a);    // выведет 0.30000000000000004, а не 0.3
```

Это фундаментальное свойство двоичного представления дробных чисел. это знать надо.

3.2. `float` — одинарная точность

Определение. `float` — 4 байта, ~7 значащих цифр. **Обязательно** с суффиксом `f` в конце литерала.

Зачем нужен. Экономия памяти в больших массивах (графика, физика). В бизнес-коде не используется.

Пример.

```
float temperature = 36.6f;    // без f — ошибка
```

3.3. `decimal` — десятичный тип

Определение. `decimal` — 16 байт, 28–29 значащих цифр, хранится **в десятичной**, а не двоичной форме. Без ошибок округления.

Зачем нужен. Деньги. Банковские расчёты. Налоги. Любая задача, где ошибка в 4-м знаке после запятой — это баг.

Пример.

```
decimal price = 99.99m;    // суффикс m обязателен
decimal tax = price * 0.20m;
Console.WriteLine(tax);    // ровно 19.998, без мусора
```

Почему double, а не decimal. Для учебной задачи разница незаметна, а decimal требует суффикса m и медленнее. В реальном коде магазина надо было бы decimal.

3.4. Таблица сравнения

Тип	Байт	Точность	Где применять
float	4	~7 знаков	Графика, физика
double	8	~15 знаков	Общие вычисления
decimal	16	28 знаков, точно	Деньги

ЛЕКЦИЯ 4. Логический тип bool

Определение. bool — логический тип, принимает **только два значения**: true или false. Без кавычек. Занимает 1 байт.

Зачем нужен. Для флагов («да/нет»), для условий if, для результатов методов-проверок.

Пример.

```
public bool IsAvailable;    // доступен ли товар
public bool CanBuy(double price) { return Balance >= price; }
```

Как используются.

```
product.IsAvailable = true;
if (product.IsAvailable) { /* ... */ }

if (customer.CanBuy(total)) { /* ... */ } else { /* ... */ }
```

Типичные ошибки.

- `product.IsAvailable = "true";` — ошибка: это строка, а не `bool`.
- `if (customer.Balance = 100)` — ошибка: `=` это присваивание, не сравнение. Нужно `==`.

Имена булевых полей и методов. Принято начинать с `Is`, `Has`, `Can`, `Should`. Это делает код читаемым: `if (customer.CanBuy(total))` читается как вопрос.

ЛЕКЦИЯ 5. Символьный и строковый типы

5.1. `char` — один символ

Определение. `char` — ровно **один** символ Unicode. Занимает 2 байта. Литерал пишется в **одинарных** кавычках.

Зачем нужен. Для отдельных символов: разделителей, первых букв, кодов.

Пример.

```
char grade = 'A';
char space = ' ';
char digit = '7';
```

Что важно. `'A'` и `"A"` — **разные вещи**. Первое — `char`, второе — `string` из одного символа. Компилятор их не путает, ты тоже не путай.

5.2. `string` — строка текста

Определение. `string` — ссылочный тип, последовательность символов. Неизменяемый (immutable): после создания строку нельзя изменить, только заменить целиком.

Зачем нужен. Для всего, что является текстом: имена, названия, сообщения.

Пример.

```
public string Name;           // в Product
public string FullName;       // в Customer
public string StoreName;      // в Store
```

```
public string City;           // в Store
public string Manufacturer; // в задании 1
```

Основные операции.

Конкатенация (+):

```
string full = "Смартфон" + " " + "Samsung";
```

Интерполяция (\$"..."):

```
string message = $"Товар: {product.Name}, цена: {product.Price} руб.";
```

Длина (Length):

```
int len = product.Name.Length; // 8 для "Смартфон"
```

Верхний/нижний регистр:

```
string upper = product.Name.ToUpper(); // "СМАРТФОН"
string lower = product.Name.ToLower(); // "смартфон"
```

Проверка на пустоту:

```
if (string.IsNullOrEmpty(product.Name)) { /* ... */ }
```

Важно — интерполяция `"..."`. Внутри `{}` может быть:

- имя поля: `{Name}` ;
- локальная переменная: `{total}` ;
- вызов метода: `{GetTotalPrice()}` ;
- выражение: `{price * quantity}` ;
- форматирование: `{price:F2}` — два знака после запятой.

Форматирование чисел. Полезно для вывода цен:

```
Console.WriteLine($"{price:F2}"); // 30000.00
Console.WriteLine($"{price:N0}"); // 30 000
Console.WriteLine($"{percent}%"); // 10%
```

Ошибки, которые встречаются чаще всего:

- `string Name = "..."` — заглавная `S`, тип называется `string` со строчной, а `String` — это класс в BCL;
- забыть `$` перед кавычкой и получить `{Name}` вместо значения;
- перенести строку на новую строку без `+`: C# не поддерживает многострочные литералы.

Многострочные строки. Если очень нужно — используй `@"..."` (verbatim) или `"""..."""` (raw string, C# 11+):

```
string text = @"Первая строка
Вторая строка";
```

ЛЕКЦИЯ 6. `var` — неявная типизация

Определение. `var` — не тип, а **ключевое слово** «пусть компилятор сам определит тип по правой части». Тип всё равно строгий, просто не написан явно.

Зачем нужен. Сокращает код, когда тип очевиден. Убирает дублирование.

Пример.

```
var store = new Store();           // компилятор понял: Store store
var product = new Product();       // Product product
var total = 100.50 + 200;          // double total
var name = "Samsung";              // string name
var flag = true;                   // bool flag
```

Ограничения.

- **Нельзя** объявить `var x;` без инициализации — компилятор не поймёт тип.
- **Нельзя** использовать `var` для полей класса. Только для локальных переменных и параметров.
- **Нельзя** менять тип: если `var x = 5;`, то дальше `x = "строка";` — ошибка.

Стиль. лучше писать типы явно (`Store store = ...`) — так какой тип используется. `var` пригодится, когда станешь уверенным.

ЛЕКЦИЯ 7. Преобразования типов

7.1. Неявное преобразование (implicit)

Определение. Компилятор сам переводит значение, если это **безопасно** — не теряется точность.

Пример.

```
int q = 5;
double price = 100.50;
double total = price * q;    // int q автоматически стал double 5.0
```

Правило «безопасности». Более узкий тип → более широкий:

```
byte → short → int → long → float → double → decimal
```

7.2. Явное преобразование (casting)

Определение. Ты сам говоришь компилятору: «я знаю, что делаю». Пишется тип в скобках.

Пример.

```
double d = 3.99;
int i = (int)d;    // i == 3, дробная часть ОТБРАСЫВАЕТСЯ (не округляется!)
```

Опасно. Если не уверен — лучше использовать методы преобразования.

7.3. Методы преобразования

Пример.

```
string s = "123";
int i = int.Parse(s);    // 123, кинет исключение, если строка не число
bool ok = int.TryParse(s, out int result); // безопасный вариант
```

Зачем нужны. При чтении данных из консоли (`Console.ReadLine()` возвращает `string`) их надо превращать в числа:

```
Console.Write("Введите цену: ");  
double price = double.Parse(Console.ReadLine());
```

7.4. Преобразование в строку

Любой тип превращается в строку через `.ToString()` или автоматически в интерполяции:

```
int q = 5;  
string s1 = q.ToString();           // "5"  
string s2 = $"{q}";                 // "5"
```

ЛЕКЦИЯ 8. Значимые и ссылочные типы

Это **ключевая** тема для понимания C#.

8.1. Значимые типы (value types)

Определение. Переменная хранит **само значение**. Все числовые типы, `bool`, `char`, `struct`, `enum` — значимые.

Где живут. Обычно в стеке (для локальных) или внутри содержащего объекта.

Пример.

```
int a = 5;  
int b = a;    // b получил КОПИЮ значения  
b = 10;  
Console.WriteLine(a); // 5 — a не изменилось!
```

8.2. Ссылочные типы (reference types)

Определение. Переменная хранит **ссылку** на объект в куче. `string`, массивы, классы (`Product`, `Customer`), интерфейсы, делегаты — ссылочные.

Пример.

```
Product p1 = new Product();  
p1.Name = "Смартфон";
```

```
Product p2 = p1;          // p2 указывает на ТОТ ЖЕ объект  
p2.Name = "Ноутбук";
```

```
Console.WriteLine(p1.Name); // "Ноутбук" — потому что p1 и p2 — один объект!
```

8.3. Почему это важно

ты пишешь:

```
Product product = new Product();  
product.PrintInfo();
```

`product` — ссылка. Когда передаёшь её в метод, метод работает с тем же объектом.

Практический вывод. Изменения полей объекта видны везде, где есть ссылка на него. Изменения локальной копии значимого типа — только в текущей области.

8.4. `string` — особый случай

`string` ссылочный, но **неизменяемый**. Поэтому выглядит как значимый:

```
string a = "Hello";  
string b = a;  
b = b + " world";    // создаётся НОВАЯ строка, а не меняется  
Console.WriteLine(a); // "Hello"
```

ЛЕКЦИЯ 9. `null` и `nullable`-типы

Определение. `null` — специальное значение «отсутствие объекта». Ссылочные типы могут быть `null`.

Пример.

```
Product product = null;    // ссылки нет
product.Name = "X";        // ОШИБКА во время выполнения: NullReferenceException
```

Зачем нужно. Обозначать «значение ещё не задано».

Проверка на null.

```
if (product != null) { /* ... */ }
if (product?.Name != null) { /* ... */ } // null-conditional, C# 6+
```

Значимые типы и null. Обычный `int` не может быть `null`. Если нужно — используй `int?` (nullable):

```
int? age = null;        // допустимо
int value = age ?? 0;    // если null, берём 0
```

Все строковые поля у нас после `new Product()` равны `null`, пока мы им что-то не присвоим. Именно поэтому в `Main` мы сначала пишем `product.Name = "Смартфон";`, а потом уже `product.PrintInfo();`.

ЛЕКЦИЯ 10. Литералы и константы

10.1. Литералы — что это

Определение. Литерал — значение, записанное прямо в коде.

Литерал	Тип	Пример
Целое	<code>int</code>	<code>42</code>
Длинное	<code>long</code>	<code>42L</code>
Дробное	<code>double</code>	<code>3.14</code>
Float	<code>float</code>	<code>3.14f</code>
Decimal	<code>decimal</code>	<code>3.14m</code>

Литерал	Тип	Пример
Логическое	bool	true , false
Строка	string	"текст"
Символ	char	'A'
Null	любой ссылочный	null

Суффиксы — зачем они. Компилятор по умолчанию считает `3.14` типом `double`. Если нужен другой тип — суффикс обязателен. ты пишешь `/ 100.0` — суффикс `.0` гарантирует `double`, чтобы деление не потеряло дробь.

10.2. const — константа

Определение. Значение, которое **нельзя изменить** после компиляции.

Пример.

```
public const double TaxRate = 0.20;
```

Дальше `TaxRate = 0.25;` — ошибка компиляции.

Зачем. Защита от случайного изменения, читаемость (имя вместо «магического числа»).

10.3. readonly — только для чтения

Определение. Поле, которое можно задать **один раз** — в объявлении или в конструкторе. После этого — менять нельзя.

Отличие от const. `const` — на этапе компиляции, `readonly` — в момент выполнения.

ЛЕКЦИЯ 11. Какие типы выбрать для типичных задач

Что описываем	Тип	Пример
Количество	int	int Quantity = 2;
Возраст	int	int Age = 25;
Цена, деньги	decimal (в учебном — double)	decimal Price = 99.99m;
Процент, коэффициент	double	double Percent = 10;
Рейтинг	double	double Rating = 4.7;
Флаг «да/нет»	bool	bool IsAvailable = true;
Имя, название	string	string Name = "Смартфон";
Один символ	char	char Grade = 'A';
Идентификатор в БД	long	long OrderId = 1000000000L;
Значение может отсутствовать	int? , string?	int? Discount = null;

ШПАРГАЛКА ТЕРМИНОВ

Общие понятия

- Тип данных** — описание того, какие значения может хранить переменная, сколько памяти занимает и какие операции допустимы. Пример: `int` — целое число, 4 байта.
- Переменная** — именованная область памяти для значения определённого типа. `int x = 5;` — переменная `x` типа `int` со значением 5.
- Поле** — переменная **внутри класса**. `public string Name;` — поле класса `Product`.
- Локальная переменная** — переменная внутри метода. Живёт только во время выполнения метода.

Параметр — переменная в скобках сигнатуры метода. `void Pay(double amount)` — `amount` параметр.

Аргумент — значение, передаваемое при вызове. `customer.Pay(1000);` — `1000` аргумент.

Литерал — значение, записанное прямо в коде: `42` , `3.14` , `"текст"` , `true` .

Константа — неизменяемое значение. `const double Pi = 3.14;` .

Инициализация — присвоение значения при объявлении. `int x = 5;` .

Классы и объекты

Класс — шаблон (чертёж) для создания объектов. Содержит поля и методы.

Объект (экземпляр) — конкретная сущность, созданная по классу через `new` .

Конструктор — специальный метод, вызывается при `new` . Инициализирует поля.

Пространство имён (namespace) — контейнер для классов. Позволяет избегать конфликта имён.

Модификатор доступа — `public` , `private` , `protected` , `internal` . Определяет, кто может обращаться к элементу.

Поле — см. выше. Хранит состояние объекта.

Метод — функция внутри класса. Может принимать параметры и возвращать значение.

Свойство (property) — комбинация поля и метода доступа (`get` / `set`). **не используется** — классические поля.

`this` — ссылка на текущий объект внутри класса.

`static` — член класса, принадлежащий самому классу, а не объекту. Пример: `Main` — статический метод.

Типы

Значимый тип (value type) — хранит значение напрямую. Примеры: `int` , `double` , `bool` , `char` , `struct` .

Ссылочный тип (reference type) — хранит ссылку на объект в куче. Примеры: `string` , классы, массивы.

Целочисленные типы — `byte` , `short` , `int` , `long` , `sbyte` , `ushort` , `uint` , `ulong` .

Вещественные типы — `float` (4 байта), `double` (8 байт), `decimal` (16 байт, точный).

Логический тип — `bool` , значения `true` / `false` .

Символьный тип — `char` , один символ Unicode.

Строковый тип — `string` , последовательность символов.

`var` — ключевое слово неявной типизации. Компилятор сам выводит тип.

`object` — базовый тип для **всех** типов C#. Может хранить значение любого типа.

Nullable-тип — тип, допускающий `null` . Обозначается `int?` , `double?` , `string?` .

Значения и преобразования

`null` — отсутствие значения у ссылочного типа.

Неявное преобразование — автоматическое, безопасное (`int` → `double`).

Явное преобразование (cast) — ручное, с указанием типа в скобках: `(int)3.99` .

Упаковка (boxing) — превращение значимого типа в `object` . Редко нужно.

Распаковка (unboxing) — обратное превращение.

`Parse` / `TryParse` — методы преобразования строки в число.

Синтаксис

Интерполяция строк — вставка значений в строку: `$"Привет, {name}"` .

Конкатенация — склейка строк через `+` .

Составное присваивание — короткая форма: `+=` , `-=` , `*=` , `/=` , `%=` .

Оператор сравнения — `==` , `!=` , `<` , `>` , `<=` , `>=` .

Логические операторы — `&&` (И), `||` (ИЛИ), `!` (НЕ).

Тернарный оператор — `условие ? значение1 : значение2` .

Комментарий — `//` до конца строки, `/* ... */` — многострочный.

Директива `using` — подключение пространства имён: `using System;` .

File-scoped namespace — современный синтаксис `namespace ShopApp;` без фигурных скобок.

Атрибут — метаданные для класса/метода, в квадратных скобках: `[Obsolete]` .

Ошибки

Ошибка компиляции — C# не пропускает код. Пример: `int x = "abc";` .

Исключение (exception) — ошибка во время выполнения. Пример: `NullReferenceException` , `FormatException` .

`NullReferenceException` — попытка обратиться к члену объекта, который равен `null` .

`FormatException` — `int.Parse("abc")` — строка не является числом.

Стандартные классы (BCL)

`Console` — работа с консолью. `Console.WriteLine(...)` , `Console.ReadLine()` .

`Math` — математические функции. `Math.Round` , `Math.Max` , `Math.Pow` , `Math.Sqrt` .

`String` (класс, не тип `string`) — статические методы для строк. `string.IsNullOrEmpty` , `string.Join` , `string.Concat` .

`Convert` — универсальные методы преобразования. `Convert.ToInt32("123")` .

Модификаторы

`public` — доступен отовсюду.

`private` — доступен только внутри класса (по умолчанию для членов класса).

`protected` — доступен классу и его наследникам.

`internal` — доступен внутри той же сборки (по умолчанию для классов верхнего уровня).

`static` — принадлежит классу, не объекту.

`readonly` — можно задать только при объявлении или в конструкторе.

`const` — константа времени компиляции.

Объектно-ориентированные понятия

Инкапсуляция — скрывание деталей реализации за публичным интерфейсом. `public` методы, `private` поля.

Наследование — создание класса на основе другого.

Полиморфизм — способность объекта с одним интерфейсом вести себя по-разному.

Абстракция — выделение существенных характеристик. Само понятие класса `Product` — это абстракция.

Композиция — «объект содержит другие объекты».

Элемент	Задачи	Тип / роль
<code>namespace ShopApp;</code>	Каждый файл	Пространство имён
<code>public class Product</code>	<code>Product.cs</code>	Объявление класса
<code>public string Name;</code>	<code>Product.cs</code>	Поле, тип <code>string</code>
<code>public double Price;</code>	<code>Product.cs</code>	Поле, тип <code>double</code>
<code>public int Quantity;</code>	<code>Product.cs</code>	Поле, тип <code>int</code>
<code>public bool IsAvailable;</code>	<code>Product.cs</code>	Поле, тип <code>bool</code>
<code>public double GetTotalPrice()</code>	<code>Product.cs</code>	Метод, возвращает <code>double</code>
<code>public void PrintInfo()</code>	Все классы	Метод, <code>void</code>
<code>new Store()</code>	<code>Program.cs</code>	Создание объекта
<code>store.StoreName = "..."</code>	<code>Program.cs</code>	Обращение к публичному полю
<code>store.PrintInfo()</code>	<code>Program.cs</code>	Вызов публичного метода
<code> \$"Товар: {Name}"</code>	Все <code>PrintInfo</code>	Интерполяция
<code>Price * Quantity</code>	<code>Product</code>	Умножение
<code>price * Percent / 100.0</code>	<code>Discount</code>	Деление, <code>100.0</code> как <code>double</code>
<code>Balance -= amount</code>	<code>Customer</code>	Составное присваивание

Элемент	Задачи	Тип / роль
<code>Balance >= price</code>	Customer	Оператор сравнения
<code>if (customer.CanBuy(total))</code>	Program	Условие с <code>bool</code>
<code>else</code>	Program	Альтернативная ветка
<code>Console.WriteLine(...)</code>	Везде	Стандартный класс

КОНТРОЛЬНЫЕ ВОПРОСЫ ПО ТИПАМ

1. Чем `int` отличается от `long` ? Когда какой использовать?
2. Почему для денег лучше `decimal` , а не `double` ?
3. В чём разница между `float` и `double` ?
4. Почему `'A'` и `"A"` — разные вещи?
5. Что произойдёт при `int x = (int)3.99; ?`
6. Чем значимый тип отличается от ссылочного? Приведи пример.
7. Что такое `null` ? Почему `int x = null; — ошибка`, а `int? x = null; — нет?`
8. Что делает ключевое слово `var` ? Можно ли объявить `var x;` без значения?
9. Зачем в выражении `price * Percent / 100.0` писать именно `100.0` , а не `100` ?
10. Какие типы значимые, а какие — ссылочными?